Отчёт по практической работе №3
Форматы данных

0. ФИО, группа

1. Ответы на вопросы

Чем отличаются текстовые и бинарные форматы?

Текстовые форматы (JSON, XML, YAML) хранят данные в виде читаемых символов, поэтому их можно открыть в обычном блокноте и легко отлаживать, но они избыточны: в каждом сообщении повторяются имена полей, скобки и теги, а разбор текста медленнее. Бинарные форматы (MessagePack, CBOR, Protobuf) кодируют данные в виде байтов, из-за чего сообщения получаются компактнее и обрабатываются быстрее, но без специальных инструментов человек их прочитать не может.

Что такое схемная эволюция?

Схемная эволюция — это возможность изменять описание структуры данных (добавлять или удалять поля), не ломая работу уже существующих систем. В Protobuf поля идентифицируются номерами, поэтому новый код читает старые данные (отсутствующее поле получает значение по умолчанию), а старый код спокойно пропускает неизвестные ему новые поля.

2. Результаты выполнения шагов

Таблица 1. Сравнение текстовых форматов

Формат	Размер (байт)	Читаемость (1-5)	Есть ли комментарии?
JSON	253	5	Нет
XML	283	3	Да (<!-- ... -->)
YAML	182	5	Да (# ...)

Размеры в таблице 1 получены из скриптов step_json.py (с отступами indent=2), step_xml.py и step_yaml.py.

Таблица 2. Сводная таблица всех форматов (вывод run_all.py)

Формат	Размер (байт)	Читаем ли человеком?	Нужна ли схема?
JSON	211	Да	Нет
XML	283	Да	Нет

Формат	Размер (байт)	Читаем ли человеком?	Нужна ли схема?
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	159	Нет	Нет
Protobuf	117	Нет	Да

Размер JSON в таблице 2 (211 байт) меньше, чем в таблице 1 (253 байта), потому что run_all.py сериализует JSON без отступов и переносов строк.

Скриншоты

1. Вывод консоли для step_json.py и step_protobuf.py:

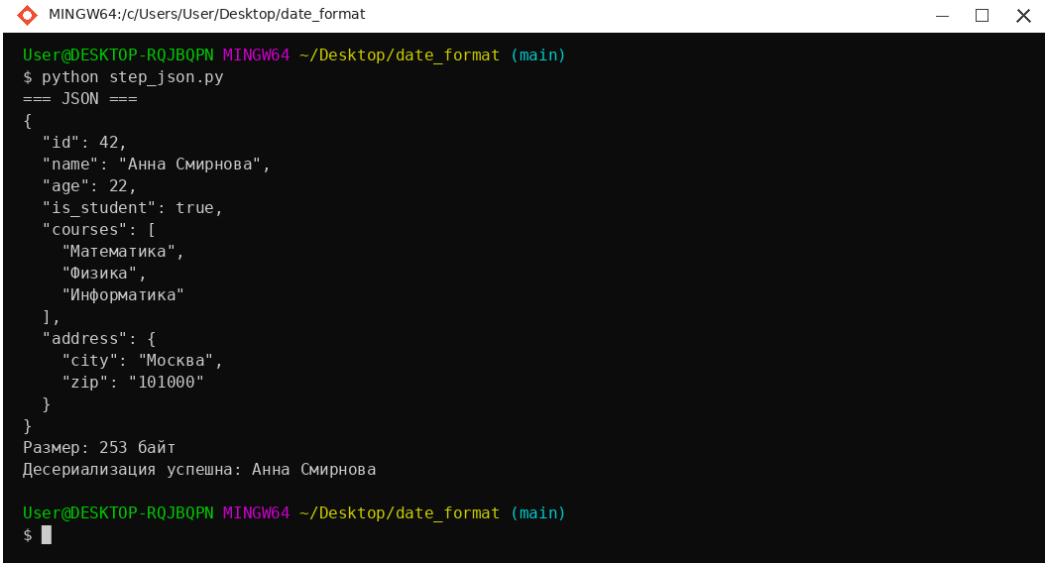


Рисунок 1 — вывод step_json.py



Рисунок 2 — вывод step_protobuf.py

2. Работа сервера, блок «=== Получено ... ===»:



Рисунок 3 — сервер получил JSON (TEXT читаемый)



Рисунок 4 — сервер получил Protobuf (TEXT — нечитаемый набор символов)

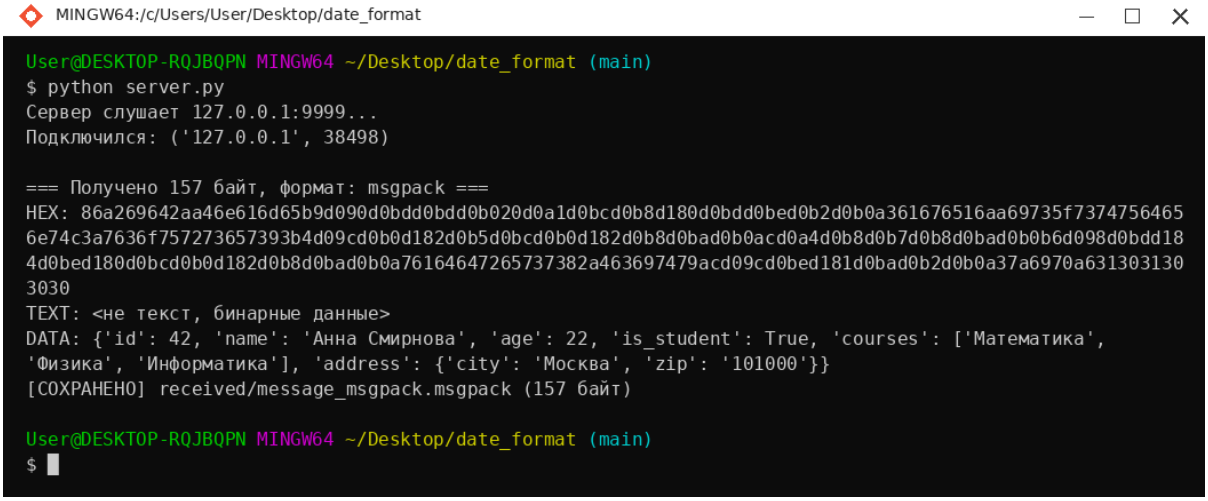


Рисунок 5 — сервер получил MessagePack (TEXT: <не текст, бинарные данные>)

Примечание: для Protobuf-сообщения этого профиля все байты случайно образуют корректную UTF-8 последовательность (кириллица кодируется в UTF-8 так же, как и в JSON, а служебные байты имеют значения меньше 0x80), поэтому сервер не выводит заглушку «<не текст, бинарные данные>»,

а печатает строку с управляющими символами. Читаемого текста при этом нет: структура данных не восстанавливается. Заглушка выводится для MessagePack и CBOR (рисунок 5).

3. Папка received/ с файлами разных форматов:

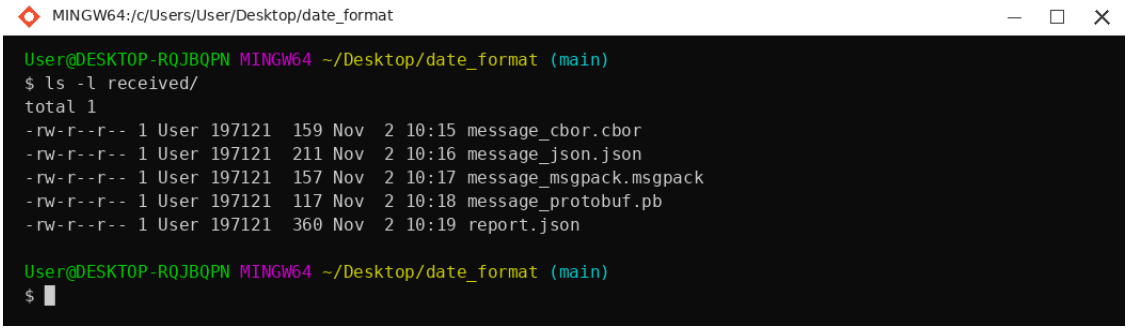


Рисунок 6 — содержимое папки received/

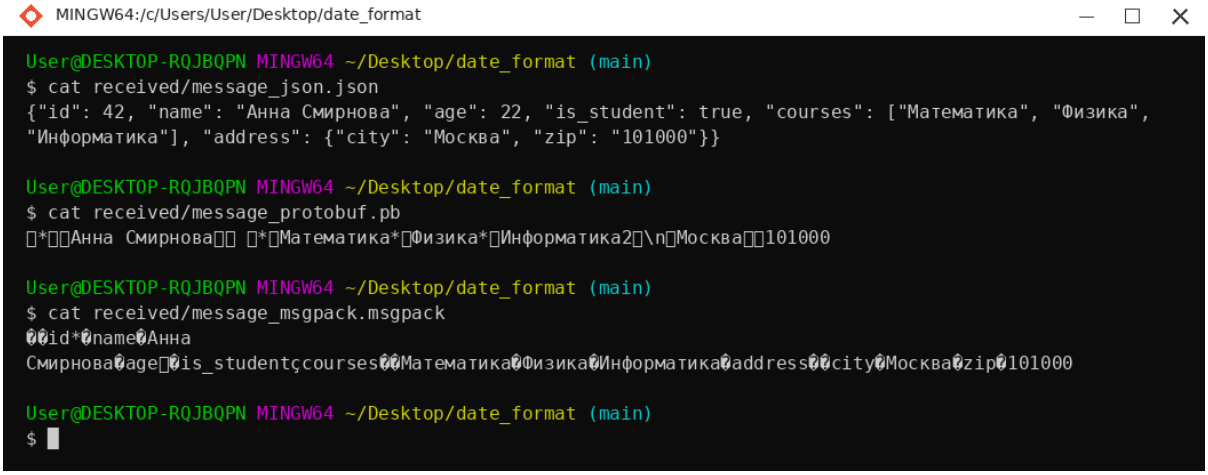


Рисунок 7 — JSON открывается как текст, а .pb и .msgpack — нет

4. Проверка схемной эволюции:

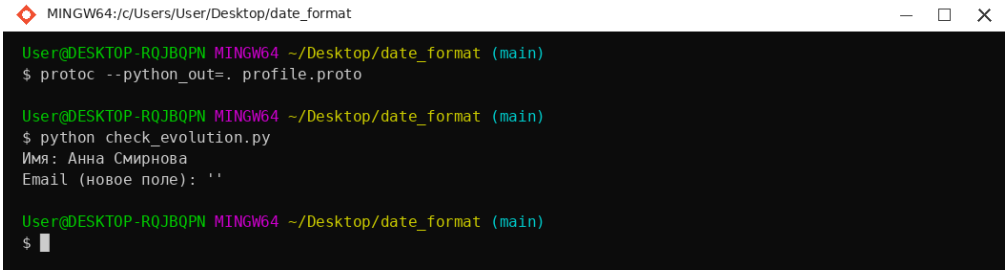


Рисунок 8 — новый код читает старый файл, поле email пустое

Для сводной статистики запущен run_all.py:

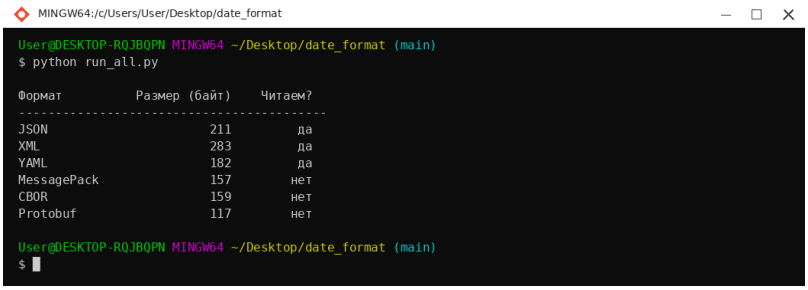


Рисунок 9 — вывод run_all.py

3. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?

В JSON каждое сообщение содержит имена полей в виде строк («name», «is_student» и т. д.), кавычки, двоеточия, запятые и пробелы, а числа и логические значения записываются текстом (true — 4 байта). В MessagePack имена полей тоже хранятся в данных, но служебные символы заменены короткими однобайтовыми типами и длинами (например, true — 1 байт, число 42 — 1 байт), поэтому получилось 157 байт против 211. В Protobuf имён полей в сообщении нет вовсе: вместо них используются номера полей из схемы (1, 2, 3...), занимающие один байт вместе с типом, поэтому размер минимальный — 117 байт, примерно на 45% меньше JSON.

2. В каком формате данные было легче всего читать вручную?

Легче всего читать YAML и JSON с отступами: структура видна по отступам, нет лишних тегов, а вложенность и списки понятны сразу. YAML лаконичнее (нет скобок и кавычек), JSON привычнее и строже. XML читать тяжелее из-за повторяющихся открывающих и закрывающих тегов. Бинарные форматы вручную не читаются вообще: в HEX-выводе угадываются только отдельные фрагменты.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

У JSON нет формальной схемы и обязательного соглашения, как обрабатывать новые, удалённые или переименованные поля: всё зависит от того, как написан код разбора, и отсутствие поля или смена типа может привести к ошибке (например, KeyError). Поля идентифицируются именами, поэтому переименование ломает совместимость, а значений по умолчанию нет. В Protobuf есть явная схема (.proto), каждое поле имеет постоянный номер, а отсутствующие поля получают значение по умолчанию. Поэтому новый код с полем email=7 прочитал старый файл без ошибок (email —

пустая строка), а старый код пропустит неизвестное поле. Главное правило: номера полей нельзя менять и повторно использовать.